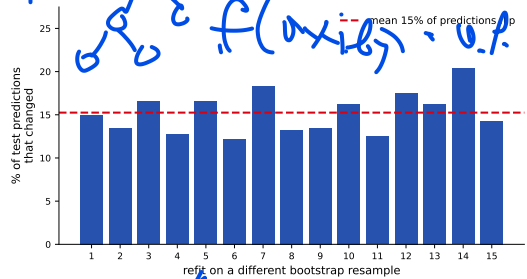
One tree is twitchy

A single decision tree ([17]) is **high variance**: refit it on a slightly different sample of the same data and it can change a lot.

Predict-first: retrain an unrestricted Titanic tree on 15 bootstrap resamples. What fraction of the *test* predictions change?



About **15%** flip – roughly one passenger in seven gets a different verdict just from re-sampling the training rows. Same method, same data size, different answers.

The plan: **average many trees** so the noise cancels. That is *bagging*, then *random forests*.

Outline

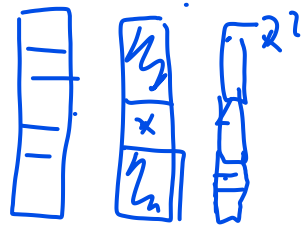
▶ Էսի սաղ ծառ ա

▶ watch on YouTube

Bagging: averaging many trees

Train many trees on resampled data, then average the noise away.

Bagging = Bootstrap AGGregating



One recipe to turn a high-variance model into a stable one:

1. **Bootstrap**: draw many resampled training sets.
2. Train one model on each.
3. **Aggregate**: combine their predictions (average / vote).

Bagging is **general** – it works for any model – but it helps most for **unstable, high-variance** learners, so **deep trees** are the perfect ingredient.

A **Random Forest** is bagging *specialized to trees* with one extra twist (Section 2). For now: just average many trees.

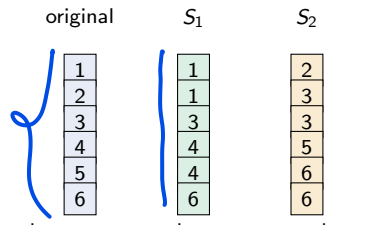
Bootstrap: resample the rows, with replacement

Reminder. A **bootstrap sample** draws rows **with replacement** (recall it from resampling / cross-validation).

In general, draw m rows from your n training rows; the classic bootstrap takes $m = n$.
Each sample comes out *slightly different*:

- some rows appear **twice or more**, others are **left out**;
- the **row mix changes** $\Rightarrow$ different counts $\Rightarrow$ different splits $\Rightarrow$ a different tree.

You can also resample **columns** (features), not just rows – that idea returns as Random Forest's per-split feature subset (Section 2).

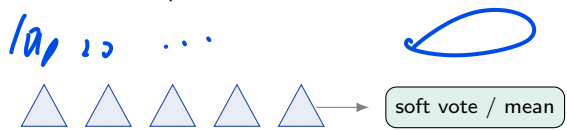


Two bootstrap samples: repeats and gaps.

Aggregate: vote or average

Classification (Titanic): each tree votes survived / died; take the **majority** – or average the predicted probabilities ("soft vote").

Regression (Yerevan rent): each tree predicts a number; take the **mean** across trees.



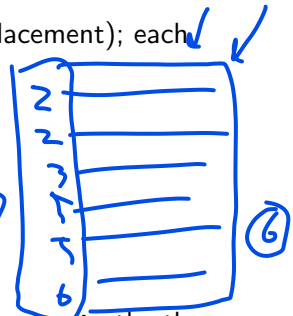
Many noisy-but-roughly-right trees, averaged, give a steadier prediction than any single one of them.

In `sklearn`, `predict` is the **soft vote**: it *averages the trees' leaf probabilities* and takes the argmax (i.e. threshold 0.5), *not* a hard tree-count majority. So `predict_proba` gives one averaged score per class – and you **threshold-tune on that** exactly as in [13].

By hand: three bootstrap samples, then aggregate

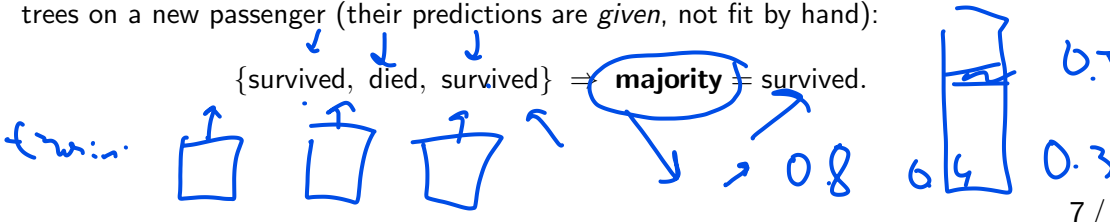
Six training rows, IDs 1–6. Draw **three** bootstrap samples (with replacement); each trains one tree. The rows left out are **out-of-bag (OOB)**:

sample	in-bag (rows drawn)	OOB (left out)
S_1	2, 2, 3, 5, 5, 6	1, 4
S_2	1, 1, 3, 4, 4, 6	2, 5
S_3	2, 3, 3, 5, 6, 6	1, 4



Each sample leaves a couple of rows out – we use that next. Now **aggregate** the three trees on a new passenger (their predictions are *given*, not fit by hand):

{survived, died, survived} $\Rightarrow$ **majority** = survived.

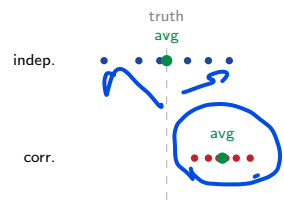


Why averaging helps – and why trees

Averaging cancels **independent** errors but not **shared** ones: trees that err in *different* directions cancel out; trees that make the *same* mistake do not.

The naive hope: average M *independent* trees (each variance σ^2) and the average has variance σ^2/M – more trees would drive it to **zero** (the $1/M$ law for a mean of independent measurements; CV's $1/k$).

But trees on the *same* data are **not** independent – their *correlated* errors put a **floor** under the variance (next frame).



Independent: avg lands on truth.
Correlated: avg stays off.

Why trees? Bagging only helps **high-variance** learners – a stable model (e.g. linear regression) has little variance to average away. So we bag **deep**, high-variance trees.

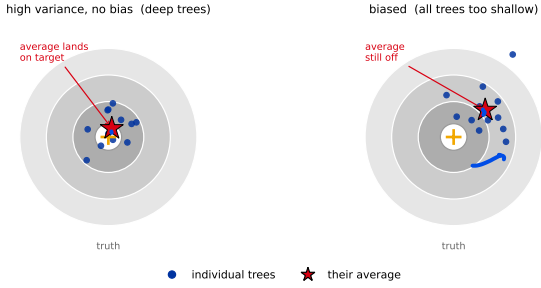
Predict-first: can averaging fix a biased model?

Suppose every tree **underfits** in the same way – all too shallow, all leaning toward the same wrong answer. Will averaging many of them fix it?

Predict-first: can averaging fix a biased model?

Suppose every tree **underfits** in the same way – all too shallow, all leaning toward the same wrong answer. Will averaging many of them fix it?

No. Averaging attacks **variance** (the scatter), not **bias** (the offset) – the darts' *average* only hits the bullseye when the trees are **unbiased**.



Bagging lowers **variance** and leaves **bias** about unchanged – which is why the formula on the next frame has *no bias term*, and why we bag *low-bias, high-variance* deep trees.

Briefly: why averaging hits a floor

We will not dwell on the algebra. Averaging M trees, each with variance σ^2 and **average pairwise correlation** ρ, gives (*The Elements of Statistical Learning*, Hastie, Tibshirani & Friedman):

$$\mathrm{Var}(\bar{f}) = \underbrace{\rho\,\sigma^2}_{\text{the floor}} + \underbrace{\frac{1-\rho}{M}\,\sigma^2}_{\text{averaging kills this}} .$$

- **Independent trees** ($\rho = 0$): only term 2 survives, so $\mathrm{Var} = \sigma^2/M \to 0$ – exactly the naive hope.
- **Identical trees** ($\rho = 1$): only term 1 survives, so $\mathrm{Var} = \sigma^2$ – averaging does **nothing**.
- **Real forests** ($0 < \rho < 1$): more trees ($M \uparrow$) kill term 2, but variance stalls at the **floor** $\rho\sigma^2$.

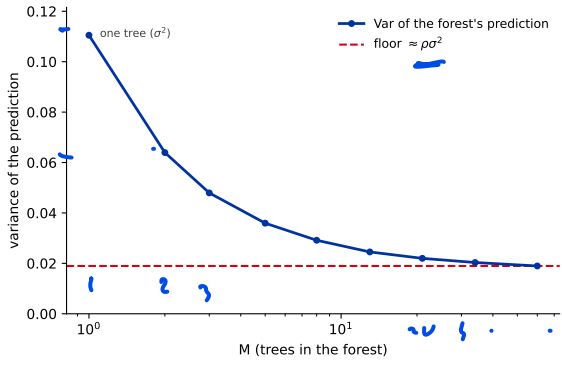
The point (not the formula): more trees help up to a point; to go further we must make the trees **less alike** – the next section.

The floor, measured

Retrain the forest on many independent training samples and track the **variance of its prediction** as trees are added:

- one tree: the full variance σ^2;
- more trees: the $(1-\rho)/M$ term shrinks, variance **drops fast**;
- then it **flattens** at the floor $\rho\sigma^2$ – extra trees buy nothing.

The two terms of the formula, now seen in data.



Why about 37% are left out: the $1/e$ limit

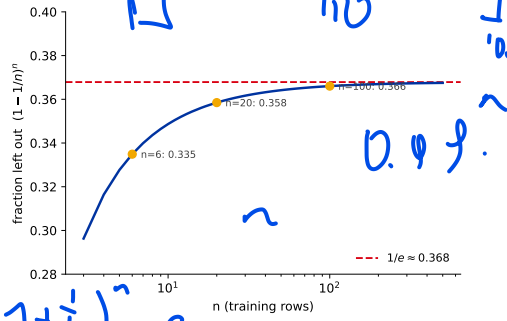
A bootstrap draws n rows **with replacement** from n rows. For one particular row:

- P(missed on one draw) $= 1 - \frac{1}{n}$.
- the n draws are independent, so P(missed on *all* n) $= \left(1 - \frac{1}{n}\right)^n$.

As n grows this is exactly the definition of $1/e$:

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow[n\to\infty]{} \frac{1}{e} \approx 0.368.$$

So ∼**37%** of rows are out-of-bag and ∼63% in-bag.

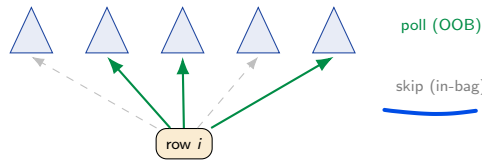


With small n the fraction is a bit lower ($n = 6 \Rightarrow \approx 33.5\%$), but it settles onto $1/e$ within a few dozen rows – which is why we just quote a flat "∼37%".

Out-of-bag (OOB) error: free validation

Each bootstrap leaves about **37%** of rows out ($(1-\frac{1}{n})^n \to \frac{1}{e}$). A row is **out-of-bag** for the trees that never saw it – so score it using **only those trees**.

- Repeat for every row → an honest held-out estimate,
- **no separate validation split needed** for tuning,
- just set `oob_score=True`.



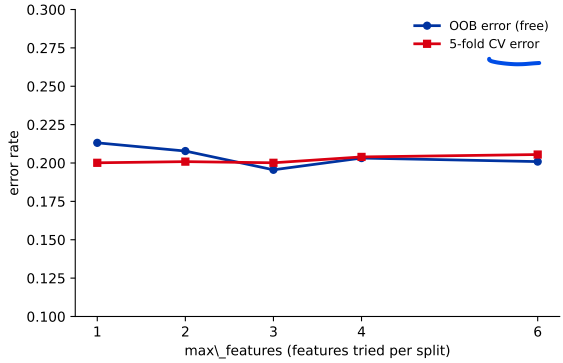
OOB error is **comparable to a held-out / CV estimate**, usually slightly *pessimistic* (each row is judged by only ∼37% of the trees). It is a property of **bagging**, not unique to RF. **But** once you *tune* against OOB it is no longer unbiased - keep an untouched test set for the final reported number.

OOB really does track cross-validation

Is OOB trustworthy? Compare it to a proper **5-fold CV** across the `max_features` knob – and OOB comes **free**, with no separate split.

The two curves nearly coincide (OOB usually a touch *pessimistic*). So you can tune on OOB and spend a real **test set** only once, for the final number.

Gotcha: the CV here **shuffles** its folds. OpenML Titanic is ordered by `pclass`, so unshuffled folds would score far too low – OOB (order-agnostic) dodges this.



From Bagging to Random Forest

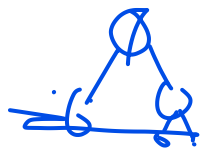
Bagged trees are too alike – force them to disagree.

The catch: bagged trees look alike

Bootstrap changes the *rows*, but every tree still sees **all the features**. So the **same strong feature** (Titanic: `gender`) tends to win the top split in nearly every tree.

Similar top splits $\Rightarrow$ similar trees $\Rightarrow$ **high** ρ $\Rightarrow$ the $\rho\sigma^2$ **floor stays high**. Averaging can only do so much.

To push the floor down we must make the trees **disagree more** – we must **decorrelate** them.



Random Forest: hide most features at each split

Random Forest (Breiman, 2001) = bagged trees where **each split** may only choose from a **random subset of F features**.

Predict-first: forcing each split to ignore most features – does that make the individual trees *better* or *worse*?

Random Forest: hide most features at each split

Random Forest (Breiman, 2001) = bagged trees where **each split** may only choose from a **random subset of F features**.

Predict-first: forcing each split to ignore most features – does that make the individual trees *better* or *worse*?

Each tree gets **slightly worse** (σ^2 up a bit). But the strong feature can no longer dominate every tree, so the trees **disagree more** (ρ down a lot).

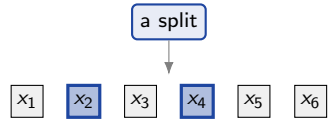
Recall the floor: $\mathrm{Var}(\bar{f}) = \underbrace{\rho\,\sigma^2}_{\text{floor}} + \frac{1-\rho}{M}\,\sigma^2$ – a smaller ρ pulls the whole floor down.

The floor $\rho\sigma^2$ drops faster than σ^2 rises – so the **average wins**: trade a little per-tree accuracy for much less correlation.

Two dice rolls: what makes a forest "random"

1. **Bootstrap the rows** (bagging, Section 1)
   - each tree trains on a different *resample* of the rows.
2. **Random feature subset per split** (the RF twist)
   - at *every* split, only F of the P features are even looked at – re-drawn fresh at each node.

Rows differ *and* the split menu differs $\Rightarrow$ trees **disagree** $\Rightarrow \rho \downarrow \Rightarrow$ the floor $\rho\sigma^2$ drops.

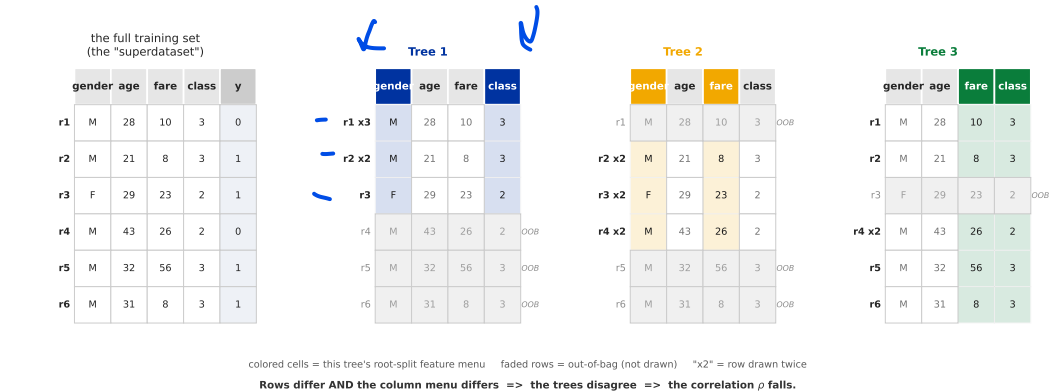


consider only $\{x_2, x_4\}$;
split on the best of them

Blue = this split's random menu; gray = hidden here.

One dataset, many different slices

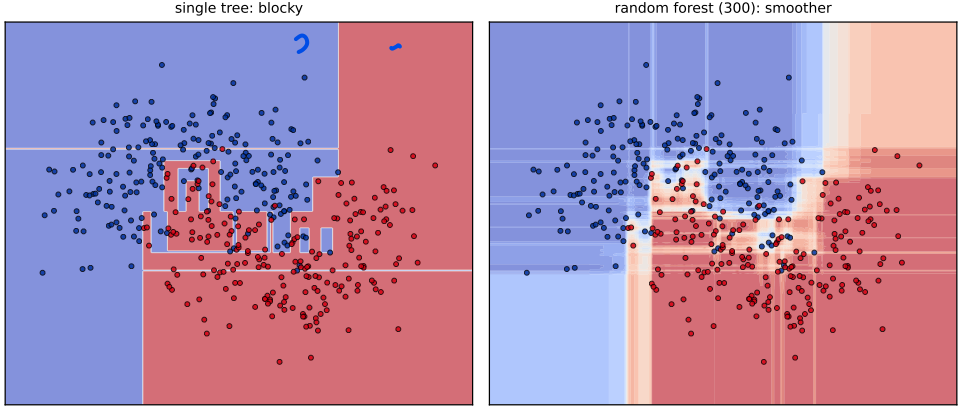
Concretely, on six Titanic rows: each tree draws its **own** bootstrap of the *rows* and, at each split, its **own** random menu of *columns*. No two trees ever train on the same slice.



The colored columns are each tree's menu **at the root split**; a *fresh* menu is re-drawn at every deeper node. (The faded out-of-bag rows are exactly what **OOB** scoring grades on – Section 1.)

What averaging buys: a smoother boundary

Same idea as the single tree's staircase in [17] – but averaging 300 trees rounds the blocky, axis-aligned steps into a softer boundary that generalizes better.



Each tree still makes boxes; *averaging* many different box-sets approximates a smooth curve.

How many features per split?

F is the decorrelation knob (sklearn: `max_features`). Rules of thumb:

- **Classification**: $F = \sqrt{P}$.
- **Regression**: $F = P/3$.
- smaller $F \Rightarrow$ more decorrelation (lower ρ), but weaker trees.

sklearn defaults, watch out: `RandomForestClassifier` uses `max_features="sqrt"` – it matches the rule. But `RandomForestRegressor` defaults to `max_features=1.0` (**all features**, since v1.1) – it does *not* apply the $P/3$ rule for you. On regression, lower it by hand.

Random Forest in Practice

How many trees, which knobs, and what it still cannot do.

Predict-first: do more trees eventually overfit?

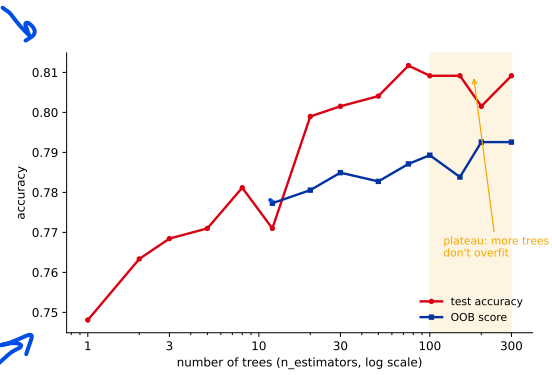
You can add hundreds of trees. Does test error eventually **rise** (overfit), like a too-deep single tree?

Predict-first: do more trees eventually overfit?

You can add hundreds of trees. Does test error eventually **rise** (overfit), like a too-deep single tree?

No. It **drops then plateaus**. More trees only shrink the $(1-\rho)/M$ term – they cannot overfit.

Precise: the *number of trees* does not overfit (the average just converges). *Deeper* trees / noisy features still can. Pick M "big enough, then stop" – you do not tune it by CV.



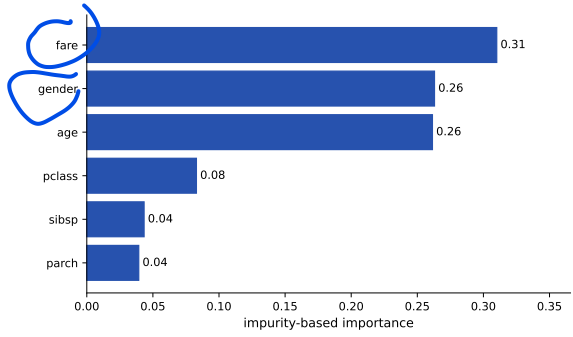
Titanic: rises, then flat past ∼100 trees. OOB (blue) starts at ∼10 trees: with fewer, some rows are out-of-bag for *no* tree, so their OOB vote is undefined.

Contrast (foreshadow [19]): in **boosting**, more trees *can* overfit – there, trees are added to correct each other, not averaged independently.

Feature importance (the default version)

A forest sums, over all trees, how much each feature's splits **dropped impurity**: `rf.feature_importances_`. A quick read of what drove the model.

On this forest `fare` and `age` outrank `gender` – the *opposite* of [17]'s single tree, where `gender` dominated.



Read with care. Impurity importance is **biased toward high-cardinality** features – continuous `fare` / `age` have many split points, so they soak up importance. **Permutation importance** and **SHAP** (the trustworthy versions) get their own lecture later.

Random forests in sklearn

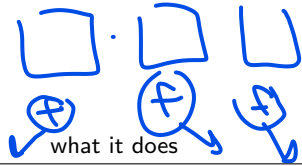
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
        n_estimators=300, max_features="sqrt",
        oob_score=True, n_jobs=-1, random_state=509)
rf.fit(X_train, y_train)      # Titanic
print(rf.oob_score_)          # free validation estimate
```

Like a single tree: **no scaling needed** (callback: preprocessing), but sklearn still needs **numeric input** – encode categoricals yourself. Use `RandomForestRegressor` for the Yerevan rent target.

In practice you often **skip sklearn** here: the boosting libraries do RF too (LightGBM `boosting_type='rf'`, XGBoost `num_parallel_tree`) – one fast library that handles `NaN` + categoricals natively and gives you tree, forest, and boosting alike (see [20] and the practical).

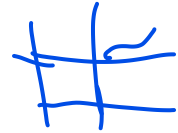
Key knobs



hyperparameter	what it does
`n_estimators`	more trees = steadier, then plateaus; costs compute. "Big enough, then stop."
`max_features`	**the decorrelation knob** (F). Lower = more diverse trees.
`max_depth` / `min_samples_leaf`	per-tree complexity (usually leave the trees deep).
`oob_score`	free held-out estimate (`=True`).
`n_jobs`	trees are independent $\Rightarrow$ trivially parallel (`=-1` uses all cores).
`class_weight`	`"balanced"` for class imbalance (callback [17]).

Extra Trees (`ExtraTreesClassifier`): picks split thresholds at *random* and, by default, **does not bootstrap** – even more randomization, faster, sometimes lower variance.

Strengths and weaknesses



Strengths

- Strong, low-tuning **default** model.
- Robust; mixed feature types, no scaling.
- **OOB** = built-in validation.
- Trees independent ⇒ trivially **parallel**.

Weaknesses

- Less **interpretable** than one tree.
- Memory / inference cost (hundreds of trees).
- **Inherited from trees**: still axis-aligned boxes ("averaging boxes gives boxes"); extrapolates poorly.

Bonus – proximities: how often two rows land in the *same leaf* across the forest is a similarity measure → outlier detection, missing-value imputation.

Aside: Isolation Forest

The forest's other job – find the odd points out, with no labels.

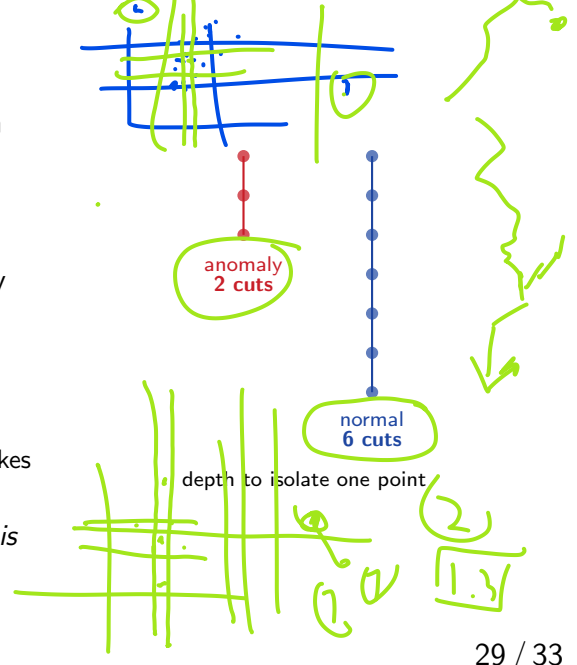
Isolation Forest: isolate, don't predict

A forest for a *different* job: **anomaly detection** with **no labels** (fraud, defects, intrusions – flag the weird points).

Grow trees by **purely random splits** (random feature, random threshold – no target). The key observation:

- an **anomaly** sits far from the crowd ⇒ a few random cuts already **isolate** it – a **short path** from the root;
- a **normal** point hides in a dense region ⇒ it takes **many** cuts to isolate – a **long path**.

So the **average path length** to isolate a point *is* the anomaly signal: **short** = **anomalous**.



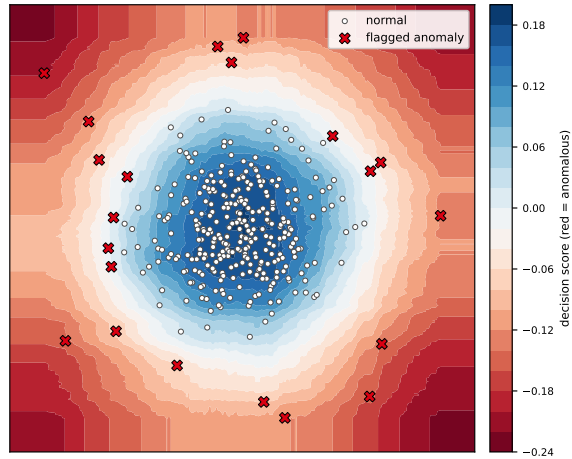
From path length to an anomaly score

Average the isolation depth $h(x)$ over all trees, normalized by $c(n)$ – the depth expected *by chance* in a tree of n points:

$$s(x) = 2^{-E[h(x)]\,/\,c(n)}.$$

- $s \to 1$: **very short** paths $\Rightarrow$ **anomaly**;
- $s \approx 0.5$: as deep as a normal point $\Rightarrow$ normal.

Each tree uses a small **subsample** (sklearn default 256 rows) – fast, and it sharpens anomalies (less "masking" by crowds).



Liu, Ting & Zhou (2008).

Isolation Forest in practice

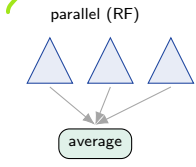
```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(n_estimators=200, contamination=0.05,
                      random_state=509)
iso.fit(X)                      # unsupervised -- no y!
flag  = iso.predict(X)          # -1 = anomaly, +1 = normal
score = iso.score_samples(X)    # lower = more anomalous
```

How it differs from a Random Forest: *unsupervised* (no target), splits are **random** rather than best-gain, and it reads **path length** instead of averaging predictions. Same ensemble-of-random-trees idea, aimed at "what is weird?" instead of "what is the label?"

Bridge: two ways to combine trees

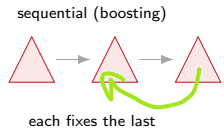
Random Forest (this deck)

- Trees grown **in parallel**, independently.
- Each sees resampled rows + a feature subset.
- **Averaging** cuts **variance**.



Boosting ([19])

- Trees grown **in sequence**.
- Each fixes the previous ones' **mistakes**.
- Sequential correction cuts **bias**.



Next ([19]): gradient boosting – and why *there*, unlike here, more trees *can* overfit.

Recap

- A single tree is **high variance**; **bagging** averages many trees on bootstrap samples to cut variance (not bias).
- Averaging cuts variance toward a **floor** set by how *correlated* the trees are; more trees alone cannot beat that floor.
- **Random Forest** = bagging + a **random feature subset per split** to lower ρ and drop that floor.
- **OOB** gives free validation; more trees **never overfit**.

Workflow: a Random Forest (`n_estimators` $\approx$ 300–500, `oob_score=True`) as a strong baseline $\rightarrow$ tune `max_features` (and depth if needed) by CV $\rightarrow$ read importances with care $\rightarrow$ reach for **boosting** ([19]) when you need more.